

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2290

**EKSPERIMENTALNA ANALIZA ENERGETSKOG PROFILA
RADA UREĐAJA OGRANIČENIH RESURSA U SUSTAVU
INTERNETA STVARI**

Nino Ljubas

Zagreb, lipanj 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2290

**EKSPERIMENTALNA ANALIZA ENERGETSKOG PROFILA
RADA UREĐAJA OGRANIČENIH RESURSA U SUSTAVU
INTERNETA STVARI**

Nino Ljubas

Zagreb, lipanj 2026.

ZAVRŠNI ZADATAK br. 2290

Pristupnik: **Nino Ljubas (0036555346)**

Studij: Elektrotehnika i informacijska tehnologija i Računarstvo

Modul: Računarstvo

Mentor: prof. dr. sc. Gordan Ježić

Zadatak: **Eksperimentalna analiza energetskog profila rada uređaja ograničenih resursa u sustavu Interneta stvari**

Opis zadatka:

Uređaji ograničenih resursa u sustavima Interneta stvari često rade u cikličkom režimu rada koji uključuje stanje mirovanja, periodično buđenje, prikupljanje podataka, lokalnu obradu te komunikaciju s nadređenim sustavom. U novijim pristupima takvim se uređajima može dodati i mogućnost lokalnog izvođenja zaključivanja modela umjetne inteligencije, što potencijalno smanjuje količinu prenesenih podataka, ali povećava računalno opterećenje i potrošnju energije. Vaš je zadatak eksperimentalno analizirati energetski profil rada uređaja ograničenih resursa kroz tipičan radni ciklus te dodatno ispitati utjecaj uključivanja lokalnog zaključivanja na ukupnu potrošnju energije. Potrebno je implementirati scenarij koji uključuje stanje mirovanja, periodično buđenje, lokalnu obradu podataka, izvođenje zaključivanja odabranog modela i komunikaciju. Na temelju mjerenja potrebno je izraditi energetski model jednog radnog ciklusa uređaja i analizirati doprinos pojedinih faza ukupnoj potrošnji.

Rok za predaju rada: 12. lipnja 2026.

SADRŽAJ

1. Uvod	1
2. Opis sustava i korištene tehnologije	3
2.1. Arhitektura sustava	3
2.2. Senzor DHT11	4
2.3. Mikrokontroler ESP32-S3	4
2.4. Povezivanje senzora i mikrokontrolera	6
2.5. Power Profiler Kit II	8
2.6. Komunikacijski protokoli (MQTT, UDP, TCP)	9
3. Razvoj sustava za predviđanje temperature	11
3.1. Konfiguracija mikrokontrolera ESP32-S3	11
3.2. Komunikacija sa senzorom DHT11	13
3.3. Implementacija TinyML modela	16
3.4. Implementacija komunikacije s nadređenim sustavom	20
4. Analiza energetskeg profila rada	26
4.1. Implementacija mjerenja potrošnje energije u sustavu	26
4.2. Izračun potrošnje energije po fazama ciklusa rada sustava ograničenih resursa	27
4.3. Utjecaj modela umjetne inteligencije na ukupnu potrošnju energije . .	28
4.4. Utjecaj komunikacijskih protokola na ukupnu potrošnju energije . . .	30
4.5. Pregled potrošnje energije po fazama ciklusa rada	34
4.6. Problemi i ograničenosti sustava	35
5. Zaključak	36
Literatura	38

Sažetak	39
Abstract	40
Izjava o korištenju umjetne inteligencije	41

1. Uvod

U tehnološkom svijetu brzih promjena i rasta tehnologija svoje mjesto zauzima i IoT (Internet of Things). Internet stvari predstavlja mrežu uređaja koji su povezani internetom i međusobno komuniciraju. Glavne ideje iza ove tehnologije su automatizacija, prikupljanje podataka i analiza u realnom vremenu. IoT je važan za mnoge industrije, od pametnih gradova i domova pa sve do zdravstva i automobilske industrije. Broj IoT uređaja se udvostručio od 2019. godine do danas, kada se procjenjuje da ih je preko 20 milijardi, a 2020. godine je broj IoT uređaja prestigao broj uređaja koji nisu IoT (Durate, 2024).

Na rubu mreže nalaze se uređaji kao što su senzori, to su uređaji ograničenih resursa, a cilj je obradu podataka lokalizirati umjesto da se svi podatci šalju prema poslužitelju. Senzori se mogu povezati s mikrokontrolerom koji je malo računalo s procesorom, memorijom te ulazno-izlaznim pinovima za povezivanje sa samim senzorom. Primjer mikrokontrolera je ESP32 serija, a ovaj rad se fokusira konkretno na rad s ESP32-S3, napredniju verziju ESP32 mikrokontrolera koju proizvodi Espressif.

Ovakav pristup računarstvu naziva se računarstvo na rubu mreže odnosno edge computing. Edge computing za cilj osim premještanja obrade podataka na uređaj ili u blizinu njega, ima i povećanje brzine jer podatci ne moraju prelaziti velike udaljenosti, što je iznimno bitno za područja koja zahtijevaju najmanju moguću latenciju, kao što su autonomna vožnja ili medicina. Također, može doći do uštede resursa, smanjenja troškova i povećanja efikasnosti. Ovaj rad se fokusira upravo na takvu analizu.

Uređaji ograničenih resursa često rade u cikličnim režimima koji se sastoje od stanja mirovanja, periodičnog buđenja, prikupljanja podataka, obrade podataka i komunikacije s nadređenim sustavom. Ovaj ciklus se može dodatno proširiti dodavanjem lokalnog izvođenja zaključivanja modela umjetne inteligencije, što može smanjiti količinu prenesenih podataka, ali isto tako povećati računalno opterećenje i potrošnju energije. Princip koji uključuje lokalno zaključivanje modela umjetne inteligencije jest Edge AI. Edge AI podrazumijeva korištenje algoritama umjetne inteligencije i AI modela koji se izvode na samom rubnom uređaju (Periyasamy, 2024). Osim smanje-

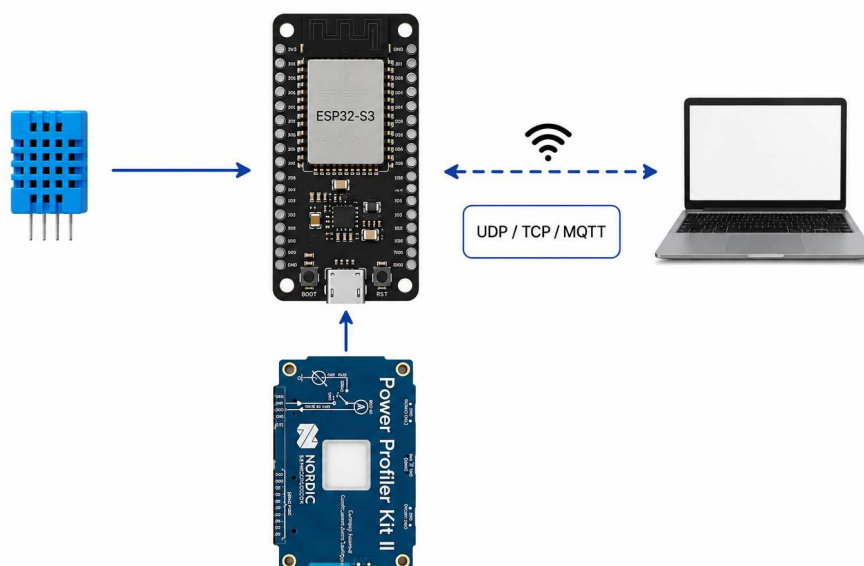
nja količine prenesenih podataka, ovakav pristup pruža i veću sigurnost jer se podatci obrađuju lokalno.

Ovaj rad za cilj ima analizirati i usporediti energetske profile rada uređaja ograničenih resursa kroz tipičan radni ciklus, te dodatno ispitati utjecaj uključivanja lokalnog zaključivanja na ukupnu potrošnju energije.

2. Opis sustava i korištene tehnologije

2.1. Arhitektura sustava

Podatci se u sustavu skupljaju preko DHT11 senzora koji periodički mjeri temperature te je povezan s ESP32-S3 mikrokontrolerom. Mikrokontroler orkestrira čitav ciklus sustava te osim veze sa senzorom izvršava zaključivanje modela umjetne inteligencije, mirovanje i komunikaciju s nadređenim sustavom. Nakon završetka zaključivanja modela umjetne inteligencije podatci se šalju prema nadređenom sustavu (prijenosnom računalu) preko Wi-Fi jednim od implementiranih komunikacijskih protokola: UDP, TCP ili MQTT. Za analizu potrošnje energije sustava korišten je Nordic Power Profiler Kit II (PPK2), koji je povezan s ESP32-S3 i služi za mjerenje potrošnje struje tijekom izvođenja pojedinih faza rada. Na slici 2.1 je opisana arhitektura sustava, pojednostavljena na prikaz isključivo komponenti i njihovih veza.



Slika 2.1: Prikaz arhitekture sustava

2.2. Senzor DHT11

DHT11 je senzor temperature i vlage koji se često koristi u IoT svijetu. Podatci senzora se mogu koristiti za analize i predikcije. Pomoću senzora moguće je pratiti uvjete u prostoru što je korisno u pametnim kućama, skladištima ili industriji. Na primjer, sustav može automatski uključiti klimu ili ventilaciju ako prepozna temperaturne anomalije u prostoriji. Temperatura se mjeri u rasponu 0°C - 50°C s odstupanjima od 2°C i vlažnost zraka u rasponu 20 - 90% s odstupanjima od 5% (Brandy, 2025). Prednost DHT11 senzora je njegova jednostavnost korištenja. Vezu s mikrokontrolerom ostvaruje samo jedan podatkovni pin. DHT senzori su također cjenovno pristupačni, što ih čini pogodnima za mnoge projekte iako nisu najprecizniji senzori koji postoje. Upravo zbog tog odnosa cijene, jednostavnosti i funkcionalnosti, često se koriste kao izbor u IoT sustavima.

U ovom sustavu obrađuju se temperature koje DHT11 mjeri i šalje mikrokontroleru. Sustav periodično prima vrijednosti izmjerene temperature u određenoj prostoriji te ovisno o implementaciji obrađuje te vrijednosti zaključivanjem umjetne inteligencije na mikrokontroleru ili ih mikrokontroler isključivo prosljeđuje dalje. Modeli umjetne inteligencije koji su korišteni trenirani su na temperaturama koje je izmjerio senzor. Mogućnost mjerenja vlage nije implementirana u ovom sustavu.

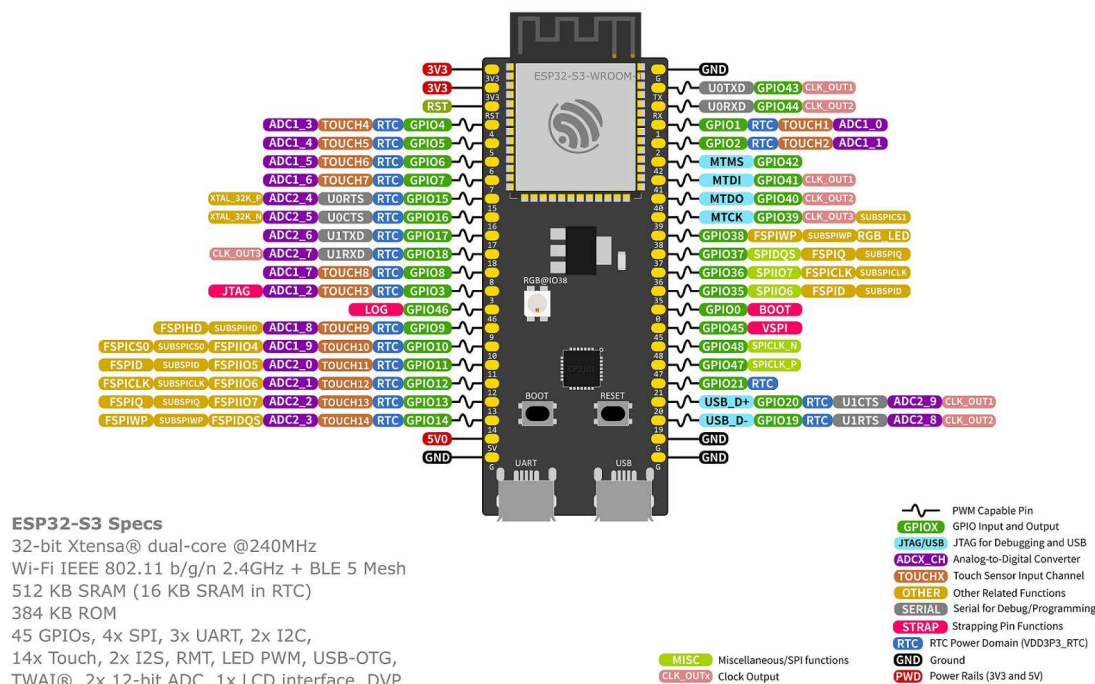
2.3. Mikrokontroler ESP32-S3

ESP32-S3 je mikrokontroler koji je razvio Espressif. Nudi visoku procesorsku snagu, integriranu Wi-Fi i Bluetooth povezivost te podršku za implementaciju modela umjetne inteligencije. U ovom potpoglavlju bit će obuhvaćene karakteristike ESP32-S3 i njegova uloga u sustavu.

ESP32-S3 je namijenjen IoT sustavima što znači da je fokus na maloj potrošnji energije, lokalnoj obradi podataka i bežičnom povezivanju. Raspolaže 32-bitnim dvojezgrenim procesorom koji može raditi na frekvenciji do 240 MHz. Moguće je povezivanje s Wi-Fi, BLE (Bluetooth Low Energy), a ovaj rad implementira Wi-Fi povezivanje. ESP32-S3 ima 512 KB SRAM (Static RAM) za pohranu programa i podataka tijekom rada. Također, SRAM je bitan za lokalnu obradu podataka kod zaključivanja modela umjetne inteligencije (Espressif, 2023). ESP32-S3 podržava više režima rada s ciljem optimizacije potrošnje energije: active mode, light sleep, deep sleep. U režimu dubokog sna, potrošnja energije je minimalna. Zbog ovakve arhitekture, ESP32-S3 je pogodan za implementaciju cikličkog rada IoT uređaja

Na slici 2.2 prikazan je raspored ulazno-izlaznih(GPIO) pinova koji se koriste za spajanje sa senzorom DHT11. Uz to, bitni su 3V3 i GND koji se koriste za napajanje i uzemljenje.

ESP32-S3-DevKitC-1



Slika 2.2: Arhitektura mikrokontrolera ESP32-S3 (Espressif, 2023)

Posebna prednost mikrokontrolera ESP32-S3 je njegova sposobnost izvođenja modela umjetne inteligencije direktno na uređaju, što ga čini pogodnim za primjenu umjetne inteligencije na rubu mreže.

Primjena umjetne inteligencije na rubu mreže mijenja princip obrade velike količine podataka na udaljenim poslužiteljima. Umjesto toga dolazi do lokalne obrade zbog mogućnosti zaključivanja modela umjetne inteligencije na mikrokontroleru. To dovodi do smanjenja količine prenesenih podataka te veće sigurnosti jer podatci ostaju lokalno što je sigurnije od slanja podataka koji mogu biti osjetljivi na poslužitelj. Uz to, veliki faktor je i potrošnja energije. Ovaj rad za cilj ima usporediti energetske profile rada sustava koji koristi zaključivanje modela umjetne inteligencije odnosno primjenjuje umjetnu inteligenciju na rubu mreže i tradicionalnog pristupa koji podatke ne obrađuje lokalno nego ih šalje na poslužitelj.

Zahvaljujući dodatnoj podršci za vektorske funkcije, ESP32-S3 omogućuje ubrzanje izvođenja neuronskih mreža. Dakle, ubrzava operacije poput množenja matrica i konvolucija te zadataka obrade signala. Korištenjem optimiziranih biblioteka kao

što su ESP-NN i ESP-DSP moguće je lokalno izvršavati modele strojnog učenja bez potrebe za povezivanje s udaljenim poslužiteljima (Espressif, 2024).

ESP32-S3 podržava implementaciju edge AI i TinyML sustava za obradu senzorskih podataka u stvarnom vremenu. Integrirane DSP i SIMD mogućnosti omogućuju učinkovito izvođenje algoritama strojnog učenja za klasifikaciju i analizu podataka dobivenih sa senzora, uključujući detekciju pokreta, prepoznavanje uzoraka, obradu audio signala i detekciju anomalija.

2.4. Povezivanje senzora i mikrokontrolera

Za povezivanje ESP32-S3 i DHT11 potrebno je 6 spojnih žica, otpornik i breadboard. Breadboard je pločica koja služi za povezivanje komponenti bez lemljenja što omogućuje jednostavne izmjene elektroničkih sklopova tijekom razvoja sustava. Sastoji se od vodljivih kontakata u koje se spajaju komponente i pinovi kako bi se spojile različite komponente. U ovom sustavu te komponente su upravo ESP32-S3 i DHT11. Spojne žice koriste se za međusobno povezivanje senzora, breadboarda i mikrokontrolera. Njihova je funkcija uspostavljanje električne veze između pinova uređaja kako bi se omogućio prijenos podataka, signala i napajanja. Kod DHT11 komunikacija ide preko jedne podatkovne linije. Pull-up otpornik je otpornik koji se koristi kako bi se ulazni pin određene komponente “povukao” na logičku razinu 1 (visoki napon) kada nema aktivnog signala. Razlog implementacije takvog otpornika je sprječavanje smetnji i nasumičnih vrijednosti. Senzor može povući liniju na LOW odnosno 0, ali ne može aktivno gurati HIGH odnosno 1. Zato je potreban pull-up otpornik između podatkovne linije i napajanja (3.3V). Dakle, otpornik kod DHT11 osigurava da je podatkovna linija u HIGH stanju kada senzor ne šalje podatke, što omogućuje ispravnu komunikaciju između senzora i mikrokontrolera.

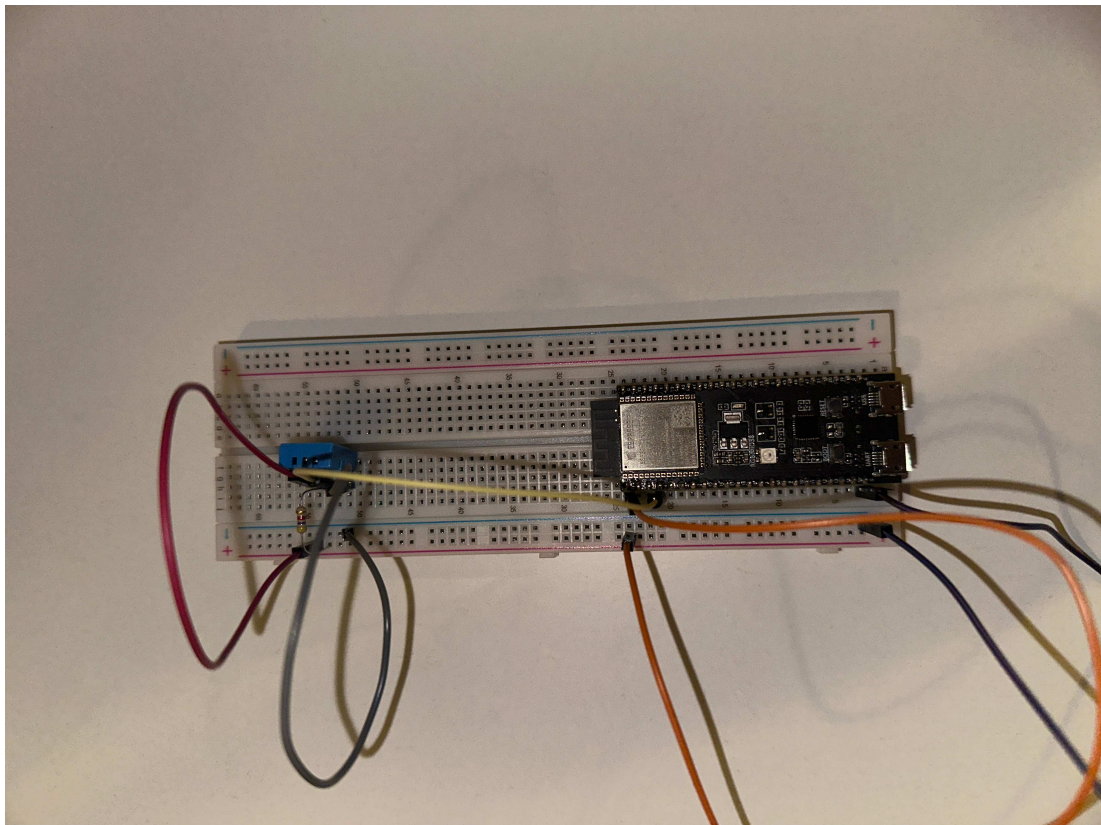
DHT11 ima 4 pina:

- VCC – napajanje 3.3V
- DATA – podatkovna linija
- NC – ne koristi se
- GND – masa

Prvi pin odnosno VCC spaja se na + (crveni stupac) na breadboard, dok se drugi pin koji se koristi za prijenos podataka spaja na mikrokontroler. Na slici 2.3 koja prikazuje na koji način su komponente povezane u ovom sustavu, žicom se povezuje drugi pin senzora (DATA) i GPIO4 na mikrokontroleru. Također, podatkovna linija se može

povezati na bilo koji GPIO mikrokontrolera. Nadalje, otpornik se povezuje između podatkovne linije (DATA) i napajanja (VCC). Treći pin se ne koristi, a četvrti pin se spaja na masu.

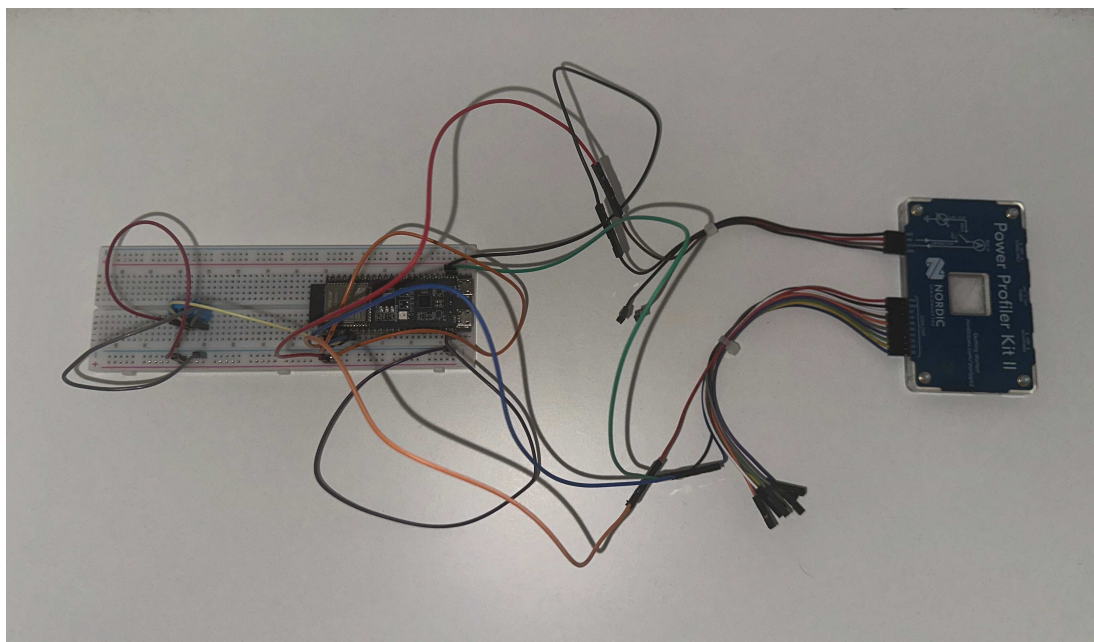
ESP32-S3 osigurava napajanje senzora putem 3.3 V izlaza, dok se zajednička masa (GND) koristi kao referentna točka za cijeli sustav. Također, omogućuje obradu i očitavanje podataka putem svojih GPIO pinova. Podatkovna linija senzora spaja se na odabrani GPIO pin (GPIO4 u ovom sustavu) te se tako ostvaruje prijenos signala koji predstavljaju izmjerene vrijednosti temperatura.



Slika 2.3: Spajanje ESP32-S3 i DHT11

2.5. Power Profiler Kit II

Za mjerenje potrošnje energije u sustavu korišten je PPK2 (Power Profile Kit II). PPK2 je precizan uređaj koji je razvio Nordic Semiconductor. Također, potrebno je preuzeti nRF Connect for Desktop i odabrati Power Profiler koji prikazuje podatke s PPK2. Kada se uređaj spoji na računalo postaje vidljiv unutar Power Profiler i moguće ga ja odabrati, a potom i mijenjati različite parametre i pratiti rezultate. Postoje dva načina rada koja se mogu odabrati. Prvi način rada je Ampere Meter Mode koji se koristi za mjerenje struje uređaja koji ima vlastito napajanje dok je drugi način Source Meter Mode koji osim što mjeri potrošnju energije daje i napajanje. U ovom sustavu korišten je Source Meter Mode te PPK2 pruža napajanje mikrokontroleru ESP32-S3. Kod korištenja tog moda potrebno je namjestiti i napon u aplikaciji odnosno 3.3V u ovom sustavu. Nakon toga, zbog jednostavnosti praćenja rada programa može se odabrati prikaz nekih od izlaza D0-D7 koji se povezuju na GPIO pinove mikrokontrolera. Sam PPK2 potrebno je povezati s mikrokontrolerom, a to se ostvaruje tako da se VOUT i GND spoje na 3.3V odnosno GND mikrokontrolera. Osim toga, spojen je D0 na GPIO5 za lakše praćenje ciklusa rada kao i VCC i GND na 3.3V odnosno GND mikrokontrolera kako bi se promjena vrijednosti D0 uopće mogla vidjeti u Power Profiler. Slika 2.4 prikazuje povezan i spreman za rad PPK2 u sustavu s ESP32-S3 i DHT11.



Slika 2.4: PPK2 u sustavu s ESP32-S3 i DHT11

2.6. Komunikacijski protokoli (MQTT, UDP, TCP)

MQTT (Message Queuing Telemetry Transport) je lagani komunikacijski protokol namijenjen uređajima ograničenih resursa. Zbog svoje jednostavnosti i male potrošnje energije odličan je protokol za IoT sustave. Temelji se na arhitekturi objavi/pretplati (publish/subscribe). Uređaji ne komuniciraju izravno, već broker koji je posrednik prima i prosljeđuje podatke. Uređaj šalje poruku na određenu temu (topic), a drugi uređaji koji trebaju te informacije pretplaćeni su na taj topic te dobivaju poruke. Protokol koristi TCP/IP te QoS (Quality of Service) odnosno definiranje pouzdanosti isporuke poruke.

MQTT definira tri osnovne QoS razine:

- QoS 0 poruka se šalje jedan put bez potvrde o primitku
- QoS 1 osigurava se primitak barem jedne kopije poruke
- QoS 2 osigurava se primitak točno jedne kopije poruke

MQTT broker korišten u ovom sustavu je Mosquitto. On implementira MQTT protokol te omogućuje komunikaciju između uređaja u publish/subscribe arhitekturi. Mosquitto je središnji posrednik koji prima poruke od uređaja i prosljeđuje ih svim pretplaćenim klijentima.

Transmission Control Protocol (TCP) je protokol koji osigurava pouzdanu komunikaciju. TCP uspostavlja vezu između različitih korisnika prije samog slanja podataka. Taj proces se naziva rukovanje te se u njemu potvrđuje spremnost za komunikaciju odnosno za slanje i primanje podataka. Uz to ovaj protokol vodi brigu da svaki paket dođe pouzdano do svog odredišta. Dakle, svaki paket ima svoj broj i potvrđuje se njegov primitak tako što primatelj šalje potvrdu (ACK). Ako paket ne dođe do primatelja na vrijeme jer je izgubljen ili previše kasni, primatelj šalje tu informaciju pošiljatelju kako bi se napravila retransmisija tog istog paketa. Na taj način primatelj prima potpunu informaciju koju je pošiljatelj poslao. Osim toga TCP implementira i druge mehanizme poput kontrole toka i kontrole zagušenja kako bi se ostvario što bolji sustav. Sve to pridonosi složenosti ovog protokola u odnosu na jednostavnije protokole kao što je UDP. Zbog toga, TCP je bolji protokol za sustave u kojima se želi točnost podataka uz gubitke na brzini zbog složenosti.

User Datagram Protocol (UDP) je protokol koji za razliku od TCP protokola ima jednostavniji način rada. UDP ne uspostavlja vezu prije samog slanja podataka te ne posjeduje mehanizam rukovanja koji osigurava da su paketi uspješno primljeni. Podatci se u ovom protokolu šalju kao datagrami za koje se ne provjerava jesu li primljeni

i u kojem rasporedu su primljeni. Zbog jednostavnijeg načina rada ima manje bitova u zaglavlju koji su potrebni za funkcioniranje protokola pa uz to što je brži ima i znatno manje opterećenje u odnosu na TCP. Navedeno UDP čini idealnim za sustave gdje se zahtjeva što manje kašnjenje i gdje je u redu da se dio podataka izgubi tijekom prijenosa. Takvi sustavi su na primjer društvene mreže s video zapisima, video pozivi, pozivi preko IP i drugi.

3. Razvoj sustava za predviđanje temperature

U ovom poglavlju prikazan je proces razvoja sustava za analizu energetskog profila rada uređaja ograničenih resursa. Obuhvaćeni su koraci konfiguracije mikrokontrolera ESP32-S3 i DHT11 senzora te implementacija TinyML modela za obradu i analizu prikupljenih podataka.

3.1. Konfiguracija mikrokontrolera ESP32-S3

Mikrokontroler ESP32-S3 konfiguriran je za periodički ciklički rad koji uključuje stanje mirovanja, prikupljanje podataka, lokalnu obradu i komunikaciju s nadređenim sustavom. Razvoj programske podrške proveden je korištenjem ESP-IDF. U ovom radu ESP-IDF je korišten za konfiguraciju Wi-Fi povezivanja, obradu podataka o temperaturi sa senzora te izvođenje TinyML modela na mikrokontroleru ESP32-S3.

ESP-IDF (Espressif IoT Development Framework) službeni je razvojni okvir za ESP32 mikrokontrolere. Pruža razvoj u programskim jezicima C i C++ te ga se može koristiti kroz VSC (Visual Studio Code) ekstenziju ili kroz terminal. ESP-IDF se može dohvatiti na više načina, a jedan od njih je kloniranje službenog GitHub repozitorija. Pri tome je potrebno nakon glavnog repozitorija klonirati i sve submodule repozitorije, što omogućava zastavica `--recursive`. Okružje dolazi s puno direktorija koji imaju primjerne programe. Pametno je koristiti neki od primjernih direktorija kao bazu kod razvoja vlastitog projekta. Takvim pristupom izbjegava se konfiguriranje stvari kao što su `CMakeLists.txt` ili `sdkconfig` što može pojednostaviti razvijanje sustava.

Da bi se omogućila konfiguracija potrebno je prvo postaviti varijable okruženja pokretanjem skripte na putanji gdje se nalazi ESP-IDF na računalu. Primjer naredbe je `. $HOME/esp-idf/export.sh`. Skripta se izvršava u trenutnoj ljusci i dodaje ESP-IDF alate u `PATH`, postavlja Python okruženje, konfigurira `toolchain` (skup alata koji pretvaraju C i C++ kod u program koji može raditi na ESP32 mikrokontroleru).

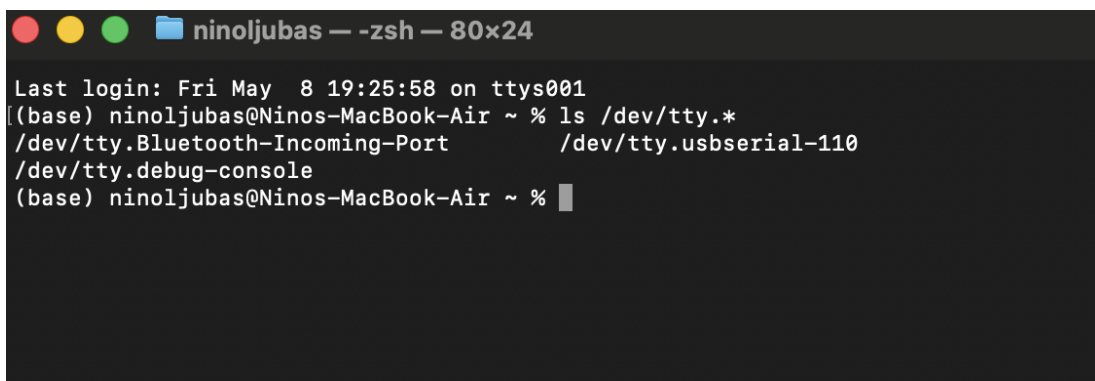
Nakon toga, omogućen je rad s `idf.py` naredbama koje bez pokretanje skripte ne bi bile prepoznate:

- `idf.py build`
- `idf.py flash`
- `idf.py monitor`
- `idf.py set-target`

Naredbom `idf.py set-target esp32s3` postavlja se mikrokontroler za koji se gradi projekt te se ta naredba pokreće iz direktorija u kojem se nalazi kod projekta. Ovaj sustav koristi ESP32-S3 te se zbog toga u `set-target` naredbi prosljeđuje ESP32-S3 te će se za njega pripremiti toolchain i odgovarajuća konfiguracija.

Zatim se naredbom `idf.py build` čita konfiguracija projekta odnosno učitava se `sdkconfig` i provjerava se odabrani mikrokontroler. Zatim se pokreće CMake i Ninja koji upravljaju procesom prevođenja projekta odnosno generiraju build sustav te određuju koje datoteke treba prevesti. Nadalje, naredba prevodi izvorni kod te povezuje sve prevedene datoteke i biblioteke u jednu cjelinu. Naposljetku, generira se ugrađeni programski kod odnosno firmware, nastaju `.bin` datoteke koje se mogu zapisati na ESP32-S3.

Naredba `ls /dev/tty.*` na Linux/macOS sustavima služi za ispis dostupnih TTY (terminalskih/serijskih) uređaja u direktoriju `/dev`. U kontekstu ESP-IDF i ESP32-S3 razvoja koristi se za pronalaženje serijskog porta na koji je spojen mikrokontroler. Primjer pokretanja naredbe prikazan je na slici 3.1 iz koje se može iščitati da je port na koji je spojen mikrokontroler `/dev/tty.usbserial-110`.



```
ninoljubas — -zsh — 80x24
Last login: Fri May  8 19:25:58 on ttys001
(base) ninoljubas@Ninos-MacBook-Air ~ % ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port      /dev/tty.usbserial-110
/dev/tty.debug-console
(base) ninoljubas@Ninos-MacBook-Air ~ %
```

Slika 3.1: ESP32-S3 port

Nakon toga, preostalo je zapisivanje ugrađenog programskog koda na mikrokontroler i pokretanje serijskog monitora za praćenje rada programa. Naredba kojom se

to ostvaruje je `idf.py -p /dev/tty.usbserial-XXXX flash monitor`. Ta naredba radi tri stvari odjednom:

- `-p /dev/tty.usbserial-XXXX` obznanijuje ESP-IDF okruženju na koji port je mikrokontroler spojen
- flash upisuje prevedeni firmware u mikrokontrolerovu flash memoriju
- monitor prikazuje ispise iz koda za praćenje rada programa u realnom vremenu

U sklopu ESP-IDF, funkcija `app_main()` predstavlja ulaznu točku programa na mikrokontroleru ESP32-S3. Za razliku od standardnih C/C++ programa koji koriste `main()`, ESP-IDF koristi vlastito izvršno okruženje odnosno runtime baziran na FreeRTOS. ESP32-S3 ne koristi klasični operativni sustav poput Windowsa ili Linuxa. Umjesto toga, programi se izvršavaju na FreeRTOS, koji je operativni sustav u realnom vremenu, optimiziran za uređaje ograničenih resursa. Prednost ovog pristupa je što program radi unutar kontroliranog višezadačnog okruženja, što omogućuje paralelno izvršavanje funkcionalnosti poput Wi-Fi komunikacije, izvođenja modela umjetne inteligencije i očitavanja senzora (Barry, 2023).

Dio koda koji postavlja konfiguraciju sustava sljedeće su funkcije:

```
nvs_flash_init();  
wifi_init();  
register_ops();
```

`nvs_flash_init()` inicijalizira NVS (Non-Volatile Storage) memoriju koja se koristi za trajno spremanje podataka mikrokontrolera. `wifi_init()` pokreće i konfigurira Wi-Fi modul mikrokontrolera ESP32-S3 dok `register_ops()` registrira dodatne operacije ili funkcionalnosti aplikacije.

3.2. Komunikacija sa senzorom DHT11

Dok se podatci između senzora i ESP32-S3 prenose podatkovnim pinom, komunikacija je implementirana s dvije funkcije koje uključuju čekanje promjene stanja pina i obradu primljenih podataka:

- `static int wait_for_state(gpio_num_t pin, int target, uint32_t timeout_us)`
- `static esp_err_t dht11_read(gpio_num_t pin, float *temp, float *hum)`

Funkcija čekanja stanja čeka logičku promjenu stanja GPIO pina. Stanje koje može poprimiti je 0 (LOW) ili 1 (HIGH). Ulaz u funkciju definiran je s tri parametra:

- `pin` je GPIO pin na koji je sustav spojen (GPIO4)
- `target` je očekivano stanje pina
- `timeout_us` vrijeme čekanja u mikrosekundama

Na početku funkcije definira se varijabla `start` koja se koristi kao referentna točka za provjeru je li prekoračen `timeout_us` odnosno provjerava se je li prekoračeno vrijeme čekanja promjene stanja pina. To je iznimno važno jer se na taj način sprječavaju beskonačne petlje u slučaju greške senzora, prekida komunikacije ili neispravnog spajanja. Funkcija zatim izvršava `while` petlju sve dok se ne promijeni vrijednost `gpio_get_level(pin)` na vrijednost varijable `target` ili dok se ne prekorači zadani `timeout_us`. Dakle, ako funkcija očekuje stanje 1, a pin je trenutno 0, petlja će nastaviti s izvođenjem. Ako vrijednost postane 1, onda petlja završava, a samim tim i funkcija.

Isječak programskog koda 3.1: Funkcija čekanja promjene stanja GPIO pina

```
static int wait_for_state(gpio_num_t pin, int target, uint32_t
    timeout_us) {
    uint32_t start = esp_timer_get_time();
    while (gpio_get_level(pin) != target) {
        if ((esp_timer_get_time() - start) > timeout_us)
            return -1;
        ets_delay_us(1);
    }
    return 0;
}
```

Funkcija `dht11_read` inicira komunikaciju sa senzorom, prima 40 bitne podatke koje sprema u polje od pet bajtova te provjerava valjanost primljenih podataka.

Na početku funkcije radi se resetiranje odnosno brisanje prethodne konfiguracije GPIO pina kako bi se moglo početi s novim čitanjem podataka. Potom se aktivira pull-up otpornik naredbom `gpio_set_pull_mode(pin, GPIO_PULLUP_ONLY);` kako bi se pin povukao u stanje 1 (HIGH) što osigurava stabilniju komunikaciju. GPIO se postavlja kao izlaz u open-drain načinu rada koji senzor očekuje naredbom `gpio_set_direction(pin, GPIO_MODE_OUTPUT_OD);` što znači da mikrokontroler može spustiti pin u stanje 0 (LOW), ali ne može podignuti u stanje 1 (HIGH). Za stanje 1 (HIGH) zaslužan je isključivo pull-up otpornik.

Senzoru je potrebno poslati startni signal koji započinje tako što mikrokontroler povuče podatkovnu liniju u stanje 0 (LOW). Navedena radnja se ostvaruje naredbom `gpio_set_level(pin, 0);` te nakon toga senzor zna da treba započeti komunikaciju. Mikrokontroler drži liniju u tom stanju 20ms, a onda se linija vraća u stanje 1 (HIGH) čime završava startni signal. Slijedi novo odgađanje od 30 mikrosekundi prije nego što se izvrši prebacivanje pina u ulazni mod. Kada postane ulaz, mikrokontroler više ne upravlja podatkovnom linijom te čeka odgovor senzora. Senzor će kao potvrdu komunikacije prvo spustiti liniju u LOW, a nakon toga podići u HIGH i ponovno spustiti u LOW. Kada se obavi ta sekvenca, senzor šalje 40 bitni podatak.

Obrada podatka izmjenjenog na senzoru izvršava se kroz petlju koja prolazi kroz svaki bit podatka. Funkcija `wait_for_state` čeka dok pin ne postane HIGH i vraća grešku ako ne dobije odgovor unutar 100µs. Ova akcija označava početak trajanja bita te se nakon nje mjeri vrijeme potrebno da pin ponovo postane LOW. Izmjereno vrijeme promjene logičkog stanja određuje vrijednost jednog bita mjerenja u petlji. Pošto mjerenje sadrži 5 bajtova potrebno je kontrolirati kojem bajtu se pristupa, a to se ostvaruju varijablom i koja se inkrementira u petlji i poljem `data[i / 8] <= 1;` koje posmiče trenutni bajt ulijevo za jedan bit. Na taj način u osam iteracija petlje odnosno u osam posmicanja se dobije bajt koji sadrži određen podatak mjerenja. Pošto posmicanjem na najmanje značajnu poziciju u bajtu dolazi nula, uvjetom `if ((esp_timer_get_time() - t0) > 40) data[i / 8] |= 1;` se na tu poziciju postavlja jedinica, ako je vrijeme trajanja bita bilo dovoljno dugo.

Isječak programskog koda 3.2: petlja za čitanje izmjenjenog podatka

```
for (int i = 0; i < 40; i++) {
    if (wait_for_state(pin, 1, 100) < 0) return ESP_FAIL;
    uint32_t t0 = esp_timer_get_time();
    if (wait_for_state(pin, 0, 100) < 0) return ESP_FAIL;
    data[i / 8] <= 1;
    if ((esp_timer_get_time() - t0) > 40) data[i / 8] |=
        1;
}
}
```

Nakon završetka petlje, potrebno je provjeriti jesu li podaci oštećeni. Struktura polja `data[]` sastoji se redom od sljedećih bajtova:

- vlaga (cijeli broj)
- vlaga (decimalni dio)

- temperatura (cijeli broj)
- temperatura (decimalni dio)
- checksum

Provjera se izvrši tako da se zbroje prva četiri elementa polja te usporede s petim. Ako je zbroj vrijednosti vlage i temperature odnosno prva četiri elementa jednak kontrolnoj sumi (checksum) odnosno petom elementu onda je izmjereni podatak ispravan, inače možemo pretpostaviti da je podatak oštećen. Ako je podatak oštećen, odbacuje se te se ispisuje poruka o grešci.

Isječak programskog koda 3.3: provjera ispravnosti podatka i ispis o nastaloj pogrešci

```
if ((uint8_t) (data[0]+data[1]+data[2]+data[3]) != data[4])
{
    ESP_LOGW(TAG, "DHT11 checksum greska");
    return ESP_FAIL;
}
```

3.3. Implementacija TinyML modela

Sustav implementira model strojnog učenja koji je prilagođen izvođenju na uređajima ograničenih resursa tako što koristi TinyML koncept u kojem je fokus upravo na uređajima na rubu mreže koji imaju ograničene resurse. To omogućuje lokalnu obradu i zaključivanje sustava izravno na ESP32-S3. Za treniranje modela koristi se TensorFlow, ali takav model nije spreman za rad na mikrokontroleru zbog svoje veličine. Korištenjem alata poput TensorFlow Lite Converter radi se optimizacija modela koji se želi učiniti manjim i prikladnijim za izvođenje na mikrokontroleru. To uključuje procese kao što je kvantizacija. U ovom radu se pretvaraju 32-bitne float vrijednosti u 8-bitne int vrijednosti čime se smanjuje veličina i ubrzava zaključivanje modela. Nakon što je model pretvoren u .tflite format, koji je optimiziran za izvođenje na uređajima s ograničenim resursima koristi se TensorFlow Lite for Microcontrollers okvir. Okvir omogućuje izvođenje modela zbog mogućnosti komunikacije s mikrokontrolerom. Dakle, učitava .tflite u flash memoriju mikrokontrolera, upravlja memorijom i implementira slojeve modela kao što je CNN ili RNN. U kontekstu TinyML sustava, često korištene arhitekture neuronskih mreža su upravo konvolucijske neuronske mreže (CNN – Convolutional Neural Networks) i rekurentne neuronske mreže (RNN – Recurrent Neural Networks).

CNN modeli su modeli dubokog učenja posebno pogodni za obradu prostornih podataka poput slika. Sastoje se od tri sloja:

- konvolucijski sloj
- pooling sloj
- potpuno povezani sloj

Konvolucijski sloj je osnovni sloj CNN modela te služi za izdvajanje značajki iz ulaznih podataka. Primjenom filtara malih dimenzija prepoznaju se obrasci pri čemu nastaju mape značajki koje se koriste u daljnjoj obradi. Pooling sloj smanjuje dimenzionalnost podataka. Na taj način se gube informacije, ali se istovremeno smanjuje kompleksnost modela i povećava efikasnost. Potpuno povezani sloj nalazi se na završnom dijelu CNN i služi za klasifikaciju podataka na temelju prethodno izdvojenih značajki. Izlaz ovog sloja predstavlja samu odluku modela.

RNN modeli dizajnirani su za obradu sekvencijalnih podataka. Za razliku od CNN, RNN modeli analiziraju ovisnost podataka kroz vrijeme jer posjeduju memoriju koja omogućava spremanje podataka iz prethodnih odlučivanja. Rekurentni sloj u RNN prenosi informacije iz prethodnog vremenskog koraka u sljedeći te se tako ostvaruje sposobnost pamćenja prethodnih podataka tijekom obrade sekvence. RNN modeli koriste se za predikciju podataka senzora kao što je senzor DHT11 i detekciju događaja temeljenih na vremenskom slijedu podataka.

Za oba modela u kodu se uključuju iste TensorFlow Lite biblioteke. Interpreter omogućuje pokretanje modela na mikrokontroleru. Uz to, učitava TensorFlow Lite modele i izvršavanje zaključivanja. Resolver služi da se definiraju operacije koje model koristi. Dakle, mikrokontroler učitava one operacije koje se definiraju u kodu, a ne sve koje postoje što omogućuje štednju memorije. Zadnji include radi interpretaciju .tflite datoteke te dohvaća verziju modela.

Isječak programskog koda 3.4: Biblioteke TensorFlow Lite Micro

```
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"
```

CNN kod ima `include "cnn_model.h"`, a RNN u svom kodu `include "rnn_model_fixed.h"`. To su C/C++ zaglavlja koja sadrže TensorFlow Lite model. Njihova svrha je ugrađivanje modela u glavni program.

U kodu se definiraju iznosi srednje vrijednosti podataka korištenih za treniranje i standardna devijacija trening podataka. Na osnovu toga se u daljnjim postupcima može

raditi normalizacija i denormalizacija temperaturnih podataka sa senzora. Ti podatci se spremaju u međuspremnik koji prima 24 vrijednosti. Na taj način se pohranjuju temperature koje se koriste kao ulazi u modele. Varijabla `temp_count` služi kao provjera je li međuspremnik koji pohranjuje temperature napunjen jer je kod pokretanja i učitavanja programa na mikrokontroler prazan.

Polje `tensor_arena` predstavlja blok RAM memorije mikrokontrolera. Ovo je potrebno jer dinamička alokacija nije moguća stoga se mora dodijeliti dovoljno memorije statički kako bi se alocirali svi tenzori te kako bi se provelo zaključivanje. U slučaju da dodijeljena memorija nije dovoljna, program se neće moći izvršiti stoga je potrebno povećavati memoriju dodijeljenu u kodu. Nadalje, upravljanje memorijom postiže se i kroz stvaranje objekta `static tflite::MicroMutableOpResolver<10> cnn_resolver;` koji služi za TensorFlow operacije CNN ili RNN modela. Broj 10 u ovom primjeru iz CNN koda govori da se koristi 10 operacija dok kada bi se operacije uključivale automatski to bi uvelo potrebu za još dodatne memorije koja se tretira pažljivo u sustavima s ograničenim resursima. U funkciji `void register_ops()` registriraju se operacije za sustav.

Glavne funkcije u kontekstu CNN i RNN su `pokreni_cnn(float* input)` i `pokreni_rnn()`. Na početku obje funkcije dohvaća se model odnosno .tflite koji dolazi iz datoteke zaglavlja proslijeđene na početku programa. Nakon toga kreira se interpreter koji ima parametre za model, resolver, arena odnosno komad RAM memorije te veličinu arene. Interpreter je potrebno alocirati, računaju se veličine tensora, raspoređuje memorija te se povezuju slojevi. U slučaju da alokacija nije uspješna događa se greška i izlaz iz navedenih funkcija, a zaključivanje modela se neće ostvariti.

Potom se dohvaća pokazivač na ulazni tenzor modela iz interpretera. Kroz objekt `input_tensor` moguće je pristupiti tipu podataka, dimenzijama i samim vrijednostima ulaznih podataka modela. Onda se provjerava koriste li se kvantizirani `int8` podatci ili `float` podatci. Ako je model tipa `kTfLiteInt8`, ulazne vrijednosti temperature prvo se normaliziraju, a onda kvantiziraju pomoću parametara `scale` i `zero_point` kako bi se prilagodile formatu modela te se upisuju u ulazni tenzor. Inače podatci se kopiraju u ulazni tenzor funkcijom `memcpy`, ako model koristi `float` vrijednosti, ali to neće biti potrebno za konkretni CNN i RNN model korišten u ovom sustavu.

Isječak programskog koda 3.5: normalizacija i kvantiziranje podataka

```
if (input_tensor->type == kTfLiteInt8) {  
    // Int8 kvantizacija  
    float scale = input_tensor->params.scale;
```

```

int zero_point = input_tensor->params.zero_point;
ESP_LOGI(TAG, "Scale: %f, ZP: %d", scale, zero_point);

for (int i = 0; i < 24; i++) {
    // normalizacija
    float normalized = (input[i] - CNN_MEAN) / CNN_STD
        ;
    int8_t q = (int8_t)((normalized / scale) +
        zero_point);
    input_tensor->data.int8[i] = q;
}
} else {
    memcpy(input_tensor->data.f, input, 24 * sizeof(float)
        );
}
}

```

Pokretanje izvođenja modela izvršava se pozivanjem funkcije unutar interpretera `TfLiteStatus s = interp.Invoke();`. Varijabla `s` pohranjuje status izvršavanja, koji može biti `kTfLiteOk` ako je zaključivanje uspješno ili grešku ako je došlo do problema. Predikcija temperature koju model napravi u ovom sustavu je kvantiziran izlaz te se vrijednost prvo dekvantizira pomoću `scale` i `zero_point`, a zatim se pretvara u normaliziranu vrijednost. Na kraju se normalizirana predikcija temperature vraća u Celzijeve stupnjeve denormalizacijom pomoću srednje vrijednosti i standardne devijacije. Vrijednost se pohranjuje u varijablu `predicted_temp_c` koja se vraća kao rezultat funkcije.

Isječak programskog koda 3.6: dohvaćanje izlaznog tenzora modela te dekvantiziranje i denormalizacija rezultata zaključivanja

```

TfLiteTensor* output_tensor = interp.output(0);
float out_norm;

if (output_tensor->type == kTfLiteInt8) {
    float scale = output_tensor->params.scale;
    int zero_point = output_tensor->params.zero_point;
    out_norm = (output_tensor->data.int8[0] - zero_point)
        * scale;
} else {
    out_norm = output_tensor->data.f[0];
}

```

```
// denormalizacija izlaza
float predicted_temp_c = out_norm * CNN_STD + CNN_MEAN;
```

Nakon pokretanja sustava, rezultati zaključivanja modela umjetne inteligencije ispisuju se u terminalu kao i vrijeme trajanja faze. Slike 3.2 i 3.3 prikazuju primjer ispisa za RNN odnosno CNN tijekom rada sustava.

```
I (33273) ZAV_RAD: [MIROVANJE] 997330 us
I (33293) ZAV_RAD: [DHT] 21664 us
I (33303) ZAV_RAD: [RNN] 2876 us → -0.02337 (norm) → 13.19 °C
I (33303) ZAV_RAD: [RNN] 9450 us
```

Slika 3.2: Ispis predviđanja RNN modela

```
I (56396) ZAV_RAD: [CNN] 2024 us → -0.63493 (norm) → 7.96 °C
I (56396) ZAV_RAD: [CNN] 4124 us
```

Slika 3.3: Ispis predviđanja CNN modela

3.4. Implementacija komunikacije s nadređenim sustavom

komunikacija između mikrokontrolera ESP32-S3 i nadređenog sustava ključna je za prijenos prikupljenih senzorskih podataka. U ovom radu ostvarena je komunikacija između ESP32-S3 mikrokontrolera i računala korištenjem tri različita protokola:

- UDP
- TCP
- MQTT

MQTT protokol je čest za IoT sustave, a temelji se na modelu publish/subscribe te posredniku (broker) koji se nalazi između. U ovom sustavu kao posrednik korišten je Mosquitto. UDP je bežični protokol koji se temelji na datagramima. Ne nudi sigurnost u dostavi informacija, ali je zato odličan za sustave koji zahtijevaju što manju latenciju te što veću brzinu. TCP pruža pouzdanu komunikaciju koja čuva originalni redoslijed poslane informacije. To ga čini zahtjevnim protokolom koji se koristi u sustavima gdje je točnost i pouzdanost važnija od same brzine komunikacije.

Kod UDP protokola prije svega je potrebno konfigurirati parametre za slanje podataka s ESP32-S3 prema nadređenom sustavu. U ovom radu za tu svrhu korišteno je osobno prijenosno računalo. Konfigurira se naziv odnosno SSID Wi-Fi mreže na

koju se mikrokontroler spaja te lozinka za pristupanje toj mreži. Nadalje, definira se IP adresa računala koje prima podatke s mikrokontrolera te vrata na kojima kojima program sluša. Navedeni podatci su preduvjet kako bi se mogli primiti UDP paketi.

Isječak programskog koda 3.7: konfiguracija parametara

```
#define TAG                "ZAV_RAD"
#define WIFI_SSID          "naziv mreze"
#define WIFI_PASS          "lozinka"
#define SERVER_IP          "IP adresa"
#define SERVER_PORT        5005
```

Nakon toga slijedi inicijalizacija i upravljanje Wi-Fi povezivanjem mikrokontrolera. Na početku se definira varijabla `wifi_events` te `WIFI_CONNECTED_BIT` koji definira prvi bit (BIT0) koji označava stanje uspješno ostvarene Wi-Fi veze. To je bitno kako bi program znao da je komunikacija spremna. Funkcija obrade mrežnih događaja obrađuje prekid veze i dobivanje IP adrese. Ako dođe do prekida veze s pristupnom točkom `WIFI_EVENT_STA_DISCONNECTED`, to će izazvati pozivanje funkcije `esp_wifi_connect()`, kako bi se ponovno uspostavila veza. Kada se primi IP adresa `IP_EVENT_STA_GOT_IP`, postavlja se bit opisan ranije unutar događajne grupe. Time se ostatku programa signalizira da je mreža dostupna.

Isječak programskog koda 3.8: funkcija obrade mrežnih događaja

```
static void wifi_event_handler(void* arg, esp_event_base_t
base, int32_t id, void* data) {
    if (base == WIFI_EVENT && id ==
        WIFI_EVENT_STA_DISCONNECTED) {
        esp_wifi_connect();
    } else if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP)
    {
        xEventGroupSetBits(wifi_events, WIFI_CONNECTED_BIT);
        ESP_LOGI(TAG, "WiFi spojen");
    }
}
```

Funkcija `wifi_init()` konfigurira parametre, pristupa bežičnoj mreži te uspostavlja vezu s pristupnom točkom. Na početku funkcije kreira se događajna grupa pomoću FreeRTOS funkcije `xEventGroupCreate()`. Događajna grupa koristi se za sinkronizaciju između različitih dijelova programa. U ovom slučaju služi za signalizaciju uspješnog povezivanja uređaja na Wi-Fi mrežu. Potom se pozivaju funkcije za inici-

jalizaciju mrežne strukture, obradu događaja te za kreiranje mrežnog sučelja u station načinu rada čime se uređaju omogućuje povezivanje na Wi-Fi mrežu kao klijent. Zatim se funkcija obrade mrežnih događaja registrira kako bi sustav mogao reagirati na promjene mrežnog stanja. Izvode se dvije registracije. Prva omogućuje obradu svih Wi-Fi događaja, a druga omogućuje obradu događaja dobivanja IP adrese. Mikrokontroler se onda postavlja u station način rada `WIFI_MODE_STA`. U tom načinu rada uređaj djeluje kao klijent koji se povezuje na usmjerivač. Nakon toga se pokreće Wi-Fi upravljački program te započinje proces povezivanja na pristupnu točku funkcijama `esp_wifi_start()`; `!esp_wifi_connect()`; . Na kraju funkcije poziva se `xEventGroupWaitBits()` odnosno funkcija koja zaustavlja izvođenje sve dok se ne postavi bit `WIFI_CONNECTED_BIT` u zadanom vremenskom intervalu. Ako se ne uspije postaviti u zadanom intervalu onda spajanje neće biti uspješno.

Slanje podataka UDP protokolom s mikrokontrolera prema računalu ostvaruje se funkcijom `udp_send(const char* msg)`. Funkcija prima jedan argument koji predstavlja niz podataka za slanje. Podatak koji se šalje sadrži informaciju o izmjerenoj temperaturi te predviđanje za sljedeću temperaturu. Prije svega se poziva `socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)` gdje ulazi redom predstavljaju korištenje IPv4, datagramski način rada te specificiranje korištenja UDP protokola. Nadalje, definira se struktura `sockaddr_in` koja sadrži informaciju o određenoj adresi dok `inet_pton()` služi za pretvorbu IP adrese u binarni oblik. Nakon toga `sendto()` šalje paket prema određenoj IP adresi i vratima s podacima.

Isječak programskog koda 3.9: funkcija za slanje podataka UDP protokolom

```
static void udp_send(const char* msg) {
    int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) return;

    struct sockaddr_in dest = {};
    dest.sin_family = AF_INET;
    dest.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, SERVER_IP, &dest.sin_addr);

    sendto(sock, msg, strlen(msg), 0,
           (struct sockaddr*)&dest, sizeof(dest));
    close(sock);

    ESP_LOGI(TAG, "UDP sent: %s", msg);
}
```

```
}
```

Na strani računala potrebno je primiti podatke s mikrokontrolera. Ovaj sustav za to koristi Python program. Prvo se definira IP adresa i vrata na kojima se očekuju podatci:

- `UDP_IP = "0.0.0.0"`
- `UDP_PORT = 5005`

Vrata su postavljena tako da njihova vrijednost odgovara prethodno definiranim vratima u kodu kako bi se mogla ostvariti komunikacija, dok navedena IP adresa prihvaća mrežni promet na svim mrežnim sučeljima uređaja umjesto na nekoj fiksnoj adresi. Zatim se kreira mrežna utičnica koja se povezuje s definiranim vratima funkcijom `sock.bind((UDP_IP, UDP_PORT))` koja rezervira odabrana vrata na računalu i omogućuje primanje podataka koji dolaze na ta vrata. Program zatim ulazi u beskonačnu petlju u kojoj prima podatke funkcijom `sock.recvfrom(1024)` koja prima do 1024 bajta i čeka pakete. Na kraju je potrebno dekodirati podatke koji su primljeni u binarnom obliku te ih pretvoriti u tekstualni oblik. Podatci koji se šalju s mikrokontrolera prema nadređenom sustavu ispisuju se pokretanjem Python programa čiji je isječak prikazan na slici 3.4. Podatci se očekuju na prethodno definiranim adresama i vratima.

```
(base) ninoljubas@Ninos-MacBook-Air code završni % python serverUDP.py
UDP server pokrenut, sluša na portu 5005...
Primljeno od ('172.20.10.4', 64127): {"temp":24.00,"rnn":13.19}
Primljeno od ('172.20.10.4', 64128): {"temp":24.00,"rnn":13.19}
```

Slika 3.4: Primanje podataka UDP protokolom na nadređenom sustavu

TCP protokol implementira Wi-Fi funkcionalnosti na isti način kao UDP. Razlika je u tome što TCP implementira funkciju `tcp_send()`. Razlika u odnosu na funkciju `udp_send()` je u tome što se prije slanja podataka uspostavlja TCP veza s poslužiteljem `connect(sock, (struct sockaddr*)dest, sizeof(dest))`. Na taj način osigurava se uspostava veze između mikrokontrolera i računala prije slanja podataka. Kada se veza uspostavi podatci se šalju funkcijom `send(sock, msg + total, len - total, 0);`. Slanje se nalazi u uvjetnoj petlji koja kontrolira da se podatak isporuči u cijelosti. Za potrebe TCP protokola također se koristi Python program. Najveća razlika u odnosu na program za UDP jest u uvjetnoj petlji koja se zaustavlja dok se ne uspostavi komunikacija. Funkcija prihvatanja `sock.accept()` nakon uspostave veze vraća objekt konekcije i adresu klijenta. Slika 3.5 prikazuje isječak pokrenutog programa za TCP protokol. Ideja je ista kao i kod UDP, ali je zbog

razlika u protokolima potrebno imati različite programe.

```
(base) ninoljubas@Ninos-MacBook-Air code završni % python serverTCP.py
TCP server pokrenut, sluša na portu 5005...
Čekam novu vezu od ESP32...
Veza uspostavljena sa: ('172.20.10.4', 54796)
Primljeno: {"temp":24.00,"rnn":13.19}
Veza zatvorena.
Čekam novu vezu od ESP32...
```

Slika 3.5: Primanje podataka TCP protokolom na nadređenom sustavu

Kod MQTT protokola uklanjaju se `SERVER_IP` i `SERVER_PORT` koji su definirali adresu i vrata računala na koje su se slali UDP i TCP paketi. Umjesto toga adresa brokera se zadaje u URI obliku. Za ovaj protokol implementiraju se dvije važne funkcije:

- `mqtt_event_handler`
- `static void mqtt_init`

Prva služi za obradu događaja odnosno praćenje stanja MQTT veze. Ovisno o trenutnom stanju u sustavu ispisuje se i odgovarajuća poruka. Ako se dogodi spajanje ispisat će se MQTT spojen na broker ili MQTT odspojen ako se dogodi odspajanje. Također, ispisuje se i MQTT error u slučaju greške. Stanje se određuje pomoću pokazivača `event_id` koji sadrži broj koji označava neki od prethodno navedenih događaja.

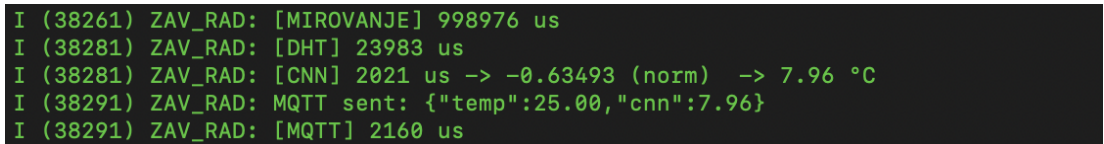
Isječak programskog koda 3.10: funkcija obrade MQTT događaja

```
static void mqtt_event_handler(void *handler_args,
    esp_event_base_t base, int32_t event_id, void *event_data)
{
    auto *event = (esp_mqtt_event_handle_t)event_data;
    switch ((esp_mqtt_event_id_t)event_id) {
        case MQTT_EVENT_CONNECTED:      ESP_LOGI(TAG, "MQTT
            spojen na broker"); break;
        case MQTT_EVENT_DISCONNECTED: ESP_LOGW(TAG, "MQTT
            odspojen");      break;
        case MQTT_EVENT_ERROR:          ESP_LOGE(TAG, "MQTT
            error");          break;
        default: break;
    }
}
```

U drugoj funkciji odvija se konfiguracija pomoću `esp_mqtt_client_config_t` `mqtt_cfg`. Prvo se koristi za postavljanje URI s IP adresom za broker. Nakon toga se

kreira klijent te `esp_mqtt_client_register_event` koji registrira prethodno navedenu obradu događaja u prvoj funkciji. Klijent se zatim pokreće i spaja na broker te održava vezu aktivnom.

Kao i kod zaključivanja modela umjetne inteligencije, informacije o slanju podataka protokolom i vrijeme trajanja faze ispisuju se u terminalu tijekom rada sustava. Slika 3.6 prikazuje jedan ciklus rada sustava koji implementira i šalje podatke MQTT protokolom kao i samo trajanje izvođenja protokola.



```
I (38261) ZAV_RAD: [MIROVANJE] 998976 us
I (38281) ZAV_RAD: [DHT] 23983 us
I (38281) ZAV_RAD: [CNN] 2021 us -> -0.63493 (norm) -> 7.96 °C
I (38291) ZAV_RAD: MQTT sent: {"temp":25.00,"cnn":7.96}
I (38291) ZAV_RAD: [MQTT] 2160 us
```

Slika 3.6: Ispis ciklusa koji implementira MQTT protokol

4. Analiza energetskeg profila rada

Ovo poglavlje je usmjereno na eksperimentalnu analizu energetskeg profila rada uređaja ograničenih resursa kroz tipičan radni ciklus. Cilj je usporediti utjecaj različitih modela umjetne inteligencije kao i različitih komunikacijskih protokola na ukupnu potrošnju energije.

4.1. Implementacija mjerenja potrošnje energije u sustavu

Na početku koda se definira `TRIGGER_PIN` `GPIO_NUM_5` na ESP32-S3 koji služi kao signal za odvijanje pojedine faze ciklusa unutar aplikacije za PPK2. Odabrani GPIO pin potrebno je prvo inicijalizirati što se postiže funkcijom koja prvo radi resetiranje pina odnosno trenutne konfiguracije, a onda postavlja smjer kao izlazni. Takav smjer je potreban kako bi mikrokontroler mogao slati nulu ili jedinicu na taj pin u određenim dijelovima ciklusa te kako bi se to onda prenijelo na PPK2. Na kraju se pin još postavi u logičku nulu odnosno u LOW stanje i ispiše se poruka o uspješno inicijaliziranom pinu u terminalu.

Isječak programskog koda 4.1: inicijalizacija pina

```
static void init_trigger_pin() {  
    gpio_reset_pin(TRIGGER_PIN);  
    gpio_set_direction(TRIGGER_PIN, GPIO_MODE_OUTPUT);  
    gpio_set_level(TRIGGER_PIN, 0);  
    ESP_LOGI(TAG, "Trigger pin GPIO%d inicijaliziran",  
              TRIGGER_PIN);  
}
```

Završni dio implementacije je upravljanje razinom pina ovisno o dijelu ciklusa u kojem se nalazi. Pin se tri puta u ciklusu diže u visoku razinu. To su slučajevi mjerenja senzora, zaključivanja modela umjetne inteligencije te komunikacija s nadređenim sus-

tavom dok je u niskoj razini za vrijeme faze mirovanja. Pin je postavljen u visoku razinu za vrijeme izvođenja tih faza pozivanjem `gpio_set_level(TRIGGER_PIN, 1)`; prije izvođenja neke od navedenih faza te `gpio_set_level(TRIGGER_PIN, 0)`; nakon završetka faze. Uz to se poziva i funkcija `esp_timer_get_time()`; također prije i nakon završetka faze kako bi se u terminalu moglo ispisati i vrijeme trajanja faze. Također, vrijeme trajanja se može bez problema odrediti i u nRF connect aplikaciji tako što se označi period u kojem je vidljivo da je d0 koji je povezan s GPIO5 bio u visokoj razini. To je zapravo period u kojem je faza trajala i može se odrediti vrijeme trajanja, ali i struja.

Isječak programskog koda 4.2: primjer implementacije mjerenja energije za senzor DHT11

```
gpio_set_level(TRIGGER_PIN, 1);
int64_t t0 = esp_timer_get_time();
esp_err_t ok = dht11_read(DHT_PIN, &temp, &hum);
int64_t t1 = esp_timer_get_time();
gpio_set_level(TRIGGER_PIN, 0);
ESP_LOGI(TAG, "[DHT] %lld us", (long long)(t1 - t0));
}
```

4.2. Izračun potrošnje energije po fazama ciklusa rada sustava ograničenih resursa

Da bi se energija izračunala potrebne su informacije o prosječnoj potrošnji električne struje, vremenski period za koji se promatra potrošnja te iznos napajanja mikrokontrolera. Te iznose može se iskoristiti u formuli koja potrošnju energije definira kao umnožak električne snage koja se može izraziti kao umnožak napona i struje te vremenskog perioda. Dakle, konačna formula glasi:

$$E = U \cdot I \cdot \Delta t \quad (4.1)$$

Iznos napona će uvijek biti 3.3V u ovom sustavu dok će iznosi prosječne električne struje i vremenski period varirati ovisno o promatranim fazama, različitim modelima umjetne inteligencije i komunikacijskim protokolima.

Ipak, dvije faze u ciklusu rada sustava ograničenih resursa će imati gotovo identične potrošnje energije jer imaju identičnu implementaciju. Faze za koje vrijedi navedeno su mjerenje senzora DHT11 i faza mirovanja koje su implementirane identično neovisno o različitim varijacijama sustava u vidu različitih modela umjetne inteligencije

ili različitih komunikacijskih protokola. Iako je implementacija za mjerenje senzora DHT11 identična za sve varijacije, postoji određena ovisnost potrošnje ovisno o komunikacijskom protokolu stoga će to biti detaljnije opisana u potpoglavlju 4.4. Faza mirovanja s druge strane traje jednu sekundu i troši prosječno oko 54mA neovisno o komunikacijskom protokolu ili modelu umjetne inteligencije. Dakle, potrošnja u toj fazi iznosi približno 178mJ. Slika 4.1 prikazuje izmjerenu prosječnu potrošnju struje u vremenu trajanja faze mirovanja.



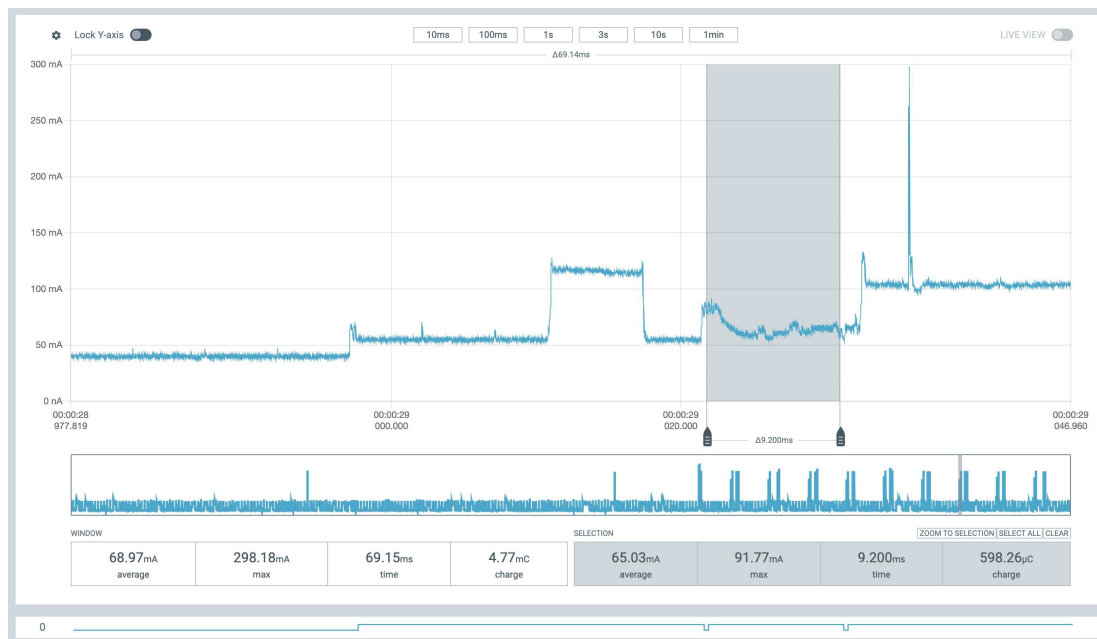
Slika 4.1: Prikaz potrošnje za fazu mirovanja

Sustav implementira `esp_wifi_set_ps(WIFI_PS_MIN_MODEM)` ; koji služi kao ušteda energije tijekom mirovanja tako što ne drži Wi-Fi radio stalno uključenim već se budi svakih 100ms kako bi provjerio upravljačke pakete koje prima. Paketi sadrže različite informacije, ali najbitnija je obavijest o tome postoje li podatkovni paketi koji su namijenjeni za uređaj u mirovanju. Na grafu je vidljivo da kod takvih buđenja postoji periodički skok potrošnje struje.

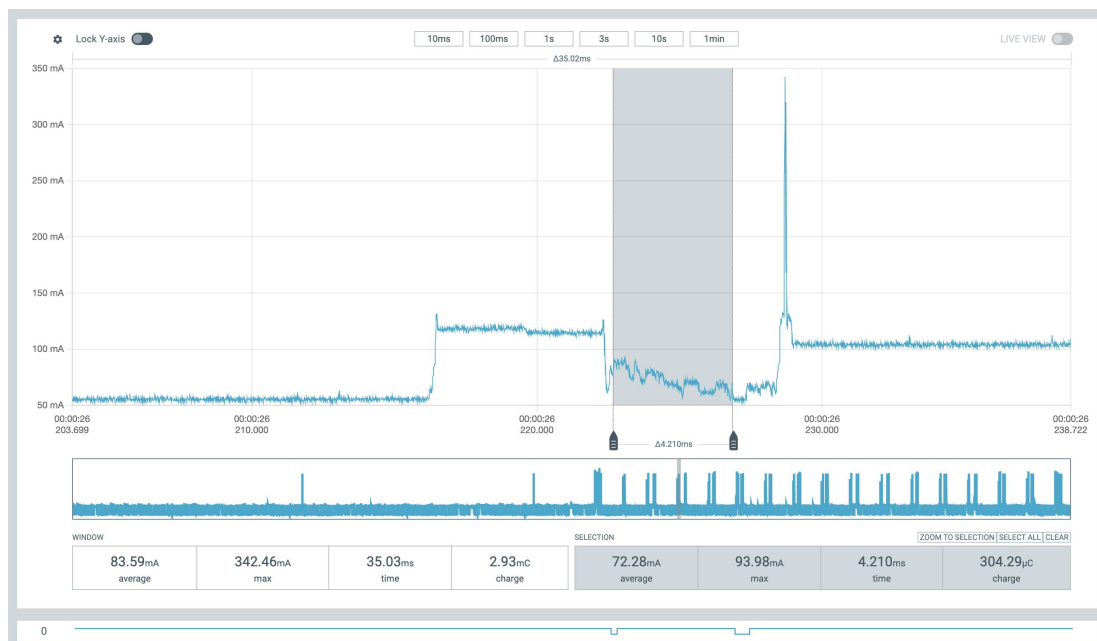
4.3. Utjecaj modela umjetne inteligencije na ukupnu potrošnju energije

Modeli umjetne inteligencije koji su korišteni za analizu su CNN i RNN. Eksperimentalnom analizom se ispostavlja da RNN ima veću potrošnju energije u odnosu na CNN. Sam poziv `interp.Invoke()` ; koji postoji u oba slučaja i pokreće zaključivanje

modela umjetne inteligencije traje oko 2ms za CNN odnosno 2.8ms za RNN. Razlika proizlazi iz arhitektura modela. Kod CNN konvolucija ima sposobnost paralelne obrade 24 uzorka temperatura u jednom prolasku dok kod RNN izlaz ovisi o prethodnom izlazu što dovodi do sekvencionalnog izvođenja. Ukupno trajanje faze pokazuje još veću razliku jer RNN zahtjeva čak 13 operacija u odnosu na definiranih 10 operacija kod CNN. S obzirom da operacije zahtijevaju vlastite alokacije to rezultira s više opterećenja na RNN model koji se ispostavlja kao složeniji model u ovom sustavu. Konačno vrijeme izvođenja kompletnih faza iznosi oko 4ms za CNN te 9ms za RNN. Potrošnja struje u ovoj fazi je usporediva za MQTT i UDP protokole te iznosi oko 62mA za RNN te 64mA za CNN. Ipak, kod TCP vrijednosti su veće te iznose 65mA za RNN i 72mA za CNN. Slike 4.2 i 4.3 prikazuju izmjerene potrošnje za RNN i CNN modele. Vidljivo je da CNN u svim implementacijama ima malo veću potrošnju. U grafovima je vidljivo da početak obje faze troši više struje u odnosu na ostatak faze što se može protumačiti količinom zadataka koje procesor mora obaviti tokom alokacije tenzora i kvantizacije na početku dok pozivanje invoke() funkcije kasnije ima nešto manju potrošnju. S obzirom da RNN ima duže trajanje izvođenja od CNN, nisu svi zadatci koje obavlja u tom periodu visoko zahtjevni što može prouzročiti nešto manju prosječnu potrošnju struje. Ukupna potrošnja energije u fazi zaključivanja modela umjetne inteligencije iznosi oko 1.94mJ za RNN i 0.86mJ za CNN kod MQTT i UDP protokola odnosno 1.97mJ za RNN i 1mJ za CNN kod TCP protokola.



Slika 4.2: Prikaz potrošnje za RNN u sustavu s TCP protokolom



Slika 4.3: Prikaz potrošnje za CNN u sustavu s TCP protokolom

Pregled izmjerenih i izračunatih vrijednosti za fazu zaključivanja modela umjetne inteligencije prikazan je u tablici 4.1. Snaga je izračunata prema izrazu $P = U \cdot I$, pri čemu je napon u svim mjerenjima iznosio 3.3V.

Tablica 4.1: Usporedba potrošnje energije za CNN i RNN modele

Model	Protokol	Invoke (ms)	Faza (ms)	Struja (mA)	Snaga (mW)	Energija (mJ)
CNN	UDP/MQTT	2,0	4,0	64	211,2	0,86
CNN	TCP	2,0	4,0	72	237,6	1,00
RNN	UDP/MQTT	2,8	9,0	62	204,6	1,94
RNN	TCP	2,8	9,0	65	214,5	1,97

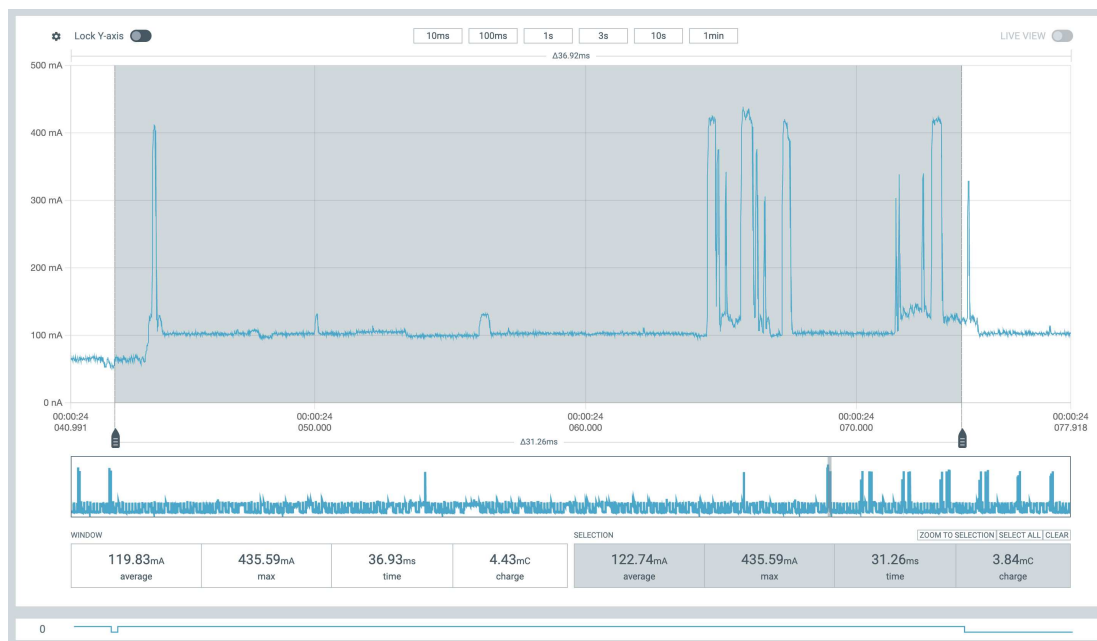
4.4. Utjecaj komunikacijskih protokola na ukupnu potrošnju energije

Analiza potrošnje energije također se provodila i za tri različita komunikacijska protokola. Kod mjerenja trigger pin je postavljen u visoko stanje prije poziva funkcije za slanje podataka, a vraćen je u nisko stanje nakon povratka iz te funkcije. Zbog toga označeni interval mjerenja u aplikaciji predstavlja vrijeme potrebno za pripremu i predaju poruke mrežnom stogu te se podatci šalju kada Wi-Fi driver i radio budu spremni.

Kod UDP protokola mjereno vrijeme se odnosi na UDP socket, poziv `sendto()` i socket zatvaranje. Kod TCP protokola također se mjeri vrijeme da se kreira socket, uspostava TCP veze s `connect()`, slanje podataka funkcijom `send()` i zatvaranje veze. Kod MQTT protokola veza s posrednikom se uspostavlja ranije, pri inicijalizaciji programa. Stoga, mjereno vrijeme se odnosi na poziv `esp_mqtt_client_publish()`, odnosno predaju poruke MQTT klijentu. Treba naglasiti da označeni interval ne mora obuhvatiti cjelokupnu aktivnost Wi-Fi radija. Kada funkcija za slanje ovih protokola vrati kontrolu programu, stvarni prijenos paketa se može ostvariti i nakon što se pin spusti u nisku razinu. U ovom radu analizirano je trajanje i potrošnja tijekom aplikacijske faze slanja. Na grafu se mogu vidjeti porasti potrošnje i nakon kraja promatranog intervala što sugerira na slanje podataka i izvan promatrane faze.

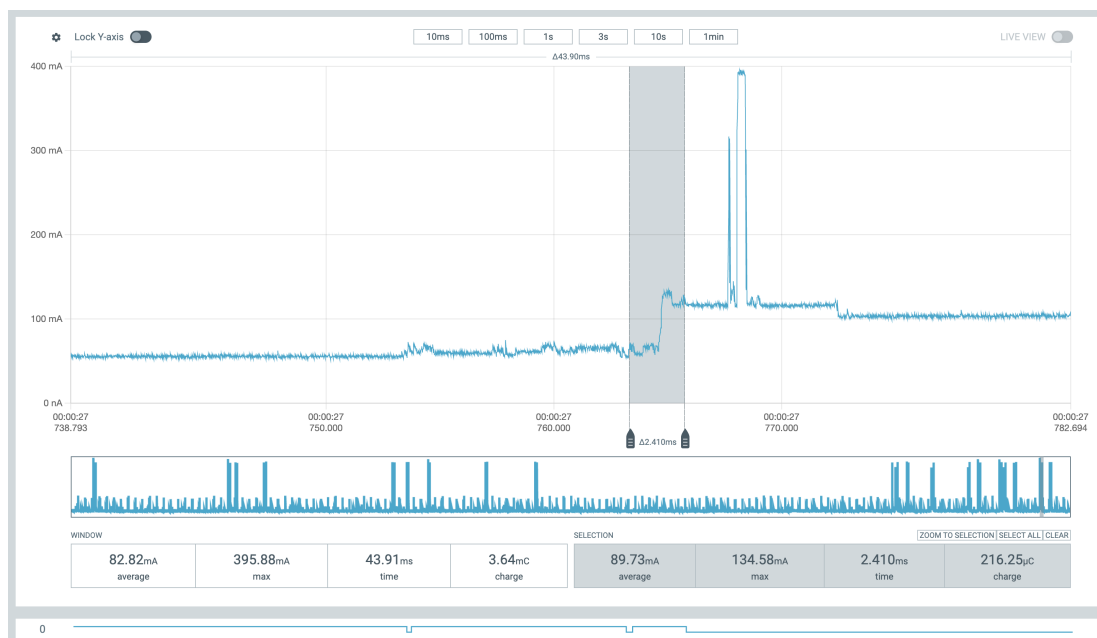
TCP u ovom radu implementira kratkotrajnu vezu jer nakon slanja zatvara vezu. Taj pristup je različit u odnosu na MQTT protokol koji održava uspostavljenu vezu s posrednikom i u jednom ciklusu samo objavljuje poruku. TCP se mogao implementirati i s trajnom vezom te bi na taj način rezultati bili sličniji MQTT protokolu. Ipak, na ovakav način se dobiva šira slika ponašanja za različite implementacije odnosno za njihove potrošnje energije s obzirom na različite implementacije sustava u realnim uvjetima. Zbog toga TCP rezultati uključuju i trošak uspostave veze, što objašnjava veće trajanje i veću potrošnju u odnosu na UDP i MQTT.

Potrošnja energije za TCP protokol varira jer TCP rukovanje ovisi o Wi-Fi stanju, mreži i retransmisijama dok UDP nema uspostavu veze, a MQTT vezu otvori jednom u `mqtt_init()`. Izmjerena trajanja za TCP variraju od nekoliko milisekundi pa sve do stotina milisekundi. Na prikazanom grafu na slici 4.4 se promatra slučaj u kojem je faza trajala oko 31ms s prosječnom strujom od oko 122mA. Potrošena energija u tom slučaju je iznosila 12.48mJ.



Slika 4.4: Prikaz potrošnje za TCP protokol

S druge strane MQTT protokol traje 2.4ms s prosječnom strujom od oko 90mA što rezultira energetsom potrošnjom iznosa 0.71mJ. UDP protokol traje oko 2.3ms s prosječnom strujom od 105mA što rezultira energetsom potrošnjom iznosa 0.79mJ. Prikaz potrošnje za oba protokola prikazan je na slikama 4.5 te 4.6.



Slika 4.5: Prikaz potrošnje za MQTT protokol



Slika 4.6: Prikaz potrošnje za UDP protokol

Na kraju, osim što komunikacijski protokoli utječu na ukupnu potrošnju energije u vlastitoj fazi, zapažena je i različita potrošnja ovisno o komunikacijskom protokolu i u ostatku ciklusa. Iako je sam kod identičan ispostavlja se da postoje određene razlike u potrošnji na primjeru faze mjerenja senzora. Samo trajanje faze mjerenja senzora DHT11 traje približno 24ms u svim implementacijama, ali prosječna potrošnja struje iznosi oko 55mA za UDP i MQTT te oko 70mA za TCP. DHT11 kod ne koristi mrežne funkcije stoga se razlika vjerojatno može pripisati različitom stanju Wi-Fi podsustava i mrežnog stoga nakon ostalih faza. Uz to, zbog kratkog trajanja faze mjerenja senzora, na izmjerenu potrošnju struje mogu utjecati periodične aktivnosti Wi-Fi radija objašnjene ranije koje se odvijaju neovisno o radu senzora. Također, izmjerena potrošnja se većinski odnosi na ESP32-S3 sustav jer dokumentacija senzora DHT11 pokazuje da njegova vlastita potrošnja tijekom mjerenja iznosi svega nekoliko miliampera. (Aosong (Guangzhou) Electronics Co., Ltd.)

U tablici 4.2 prikazana je usporedba trajanja, prosječne struje, prosječne snage i potrošene energije za fazu slanja podataka različitim komunikacijskim protokolima.

Tablica 4.2: Usporedba potrošnje energije za komunikacijske protokole

Protokol	Vrijeme (ms)	Struja (mA)	Snaga (mW)	Energija (mJ)
UDP	2,3	105	346,5	0,79
MQTT	2,4	90	297,0	0,71
TCP	31,0	122	402,6	12,48

4.5. Pregled potrošnje energije po fazama ciklusa rada

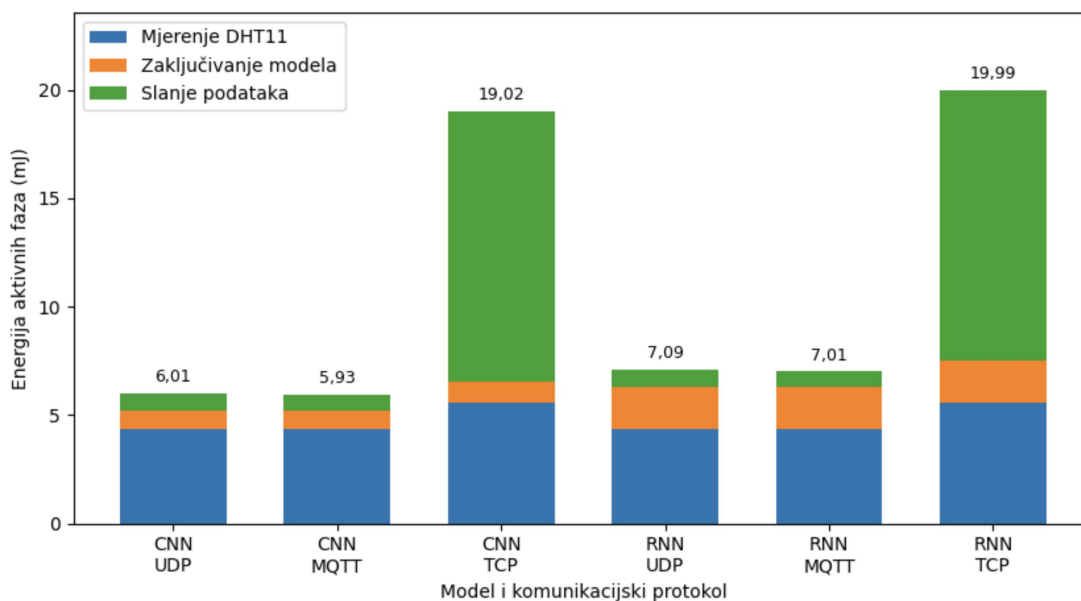
Nakon analize svih implementiranih modela umjetne inteligencije i komunikacijskih protokola sustava, u tablici 4.3 prikazan je pregled potrošnje energije po fazama ciklusa rada. Tablica uključuje trajanje faze, prosječnu struju, prosječnu snagu i potrošenu energiju.

Tablica 4.3: Pregled potrošnje energije po fazama ciklusa rada sustava

Faza	Varijanta	t (ms)	I (mA)	P (mW)	E (mJ)
Mirovanje	sve	1000,0	54	178,2	178,20
Mjerenje DHT11	UDP/MQTT	24,0	55	181,5	4,36
Mjerenje DHT11	TCP	24,0	70	231,0	5,54
Zaključivanje CNN	UDP/MQTT	4,0	64	211,2	0,86
Zaključivanje CNN	TCP	4,0	72	237,6	1,00
Zaključivanje RNN	UDP/MQTT	9,0	62	204,6	1,94
Zaključivanje RNN	TCP	9,0	65	214,5	1,97
Slanje podataka	UDP	2,3	105	346,5	0,79
Slanje podataka	MQTT	2,4	90	297,0	0,71
Slanje podataka	TCP	31,0	122	402,6	12,48

Iz tablice je vidljivo da faza mirovanja ima najveću potrošnju energije u ciklusu. Razlog nije velika prosječna struja, nego trajanje faze od jedne sekunde. Kako bi se jasnije prikazale razlike između različitih implementacija sustava na slici 4.7 prikazan je graf potrošnje aktivnog dijela ciklusa sustava.

Na slici 4.7 vidljivo je da TCP protokol značajno povećava potrošnju energije kod oba modela. Kod UDP i MQTT protokola razlika između CNN i RNN implementacija najviše proizlazi iz faze lokalnog zaključivanja modela, pri čemu RNN ima veću potrošnju zbog duljeg trajanja izvođenja i nešto veće prosječne potrošnje struje. MQTT ima nešto manju potrošnju od UDP protokola u fazi slanja podataka, ali je razlika neznatna u odnosu na TCP.



Slika 4.7: Potrošnja energije aktivnog dijela ciklusa po fazama

4.6. Problemi i ograničenosti sustava

Kod komunikacijskih protokola mjereni interval obuhvaća samo aplikacijski dio slanja. Prijenos paketa može se nastaviti i nakon vraćanja kontrole programu zbog asinkronog načina rada. Dakle, izmjerena srednja struja i energija unutar faze komunikacije s nadređenim sustavom ne predstavlja nužno cjelokupnu potrošnju.

Kratke faze poput mjerenja senzora DHT11 mogu unutar intervala trajanja dobiti Wi-Fi opterećenje koje može dodatno povisiti potrošnju energije koja je neovisna o samom mjerenju. Primjer je situacija u kojoj mjerenje senzora DHT11 ima prosječnu potrošnju struje oko 70mA za TCP odnosno oko 55mA za UDP ili MQTT protkol.

Na početku faze zaključivanja modela umjetne inteligencije uočen je kratkotrajni porast potrošnje struje. Implementacija sustava za svaki ciklus ponovno stvara objekt MicroInterpreter i poziva funkcije AllocateTensors(), može se pretpostaviti da skok na početku faze dolazi zbog inicijalizacije TensorFlow Lite Micro okruženja i pripreme ulaznih tenzora. Jedno od mogućih poboljšanja sustava bilo bi korištenje interpretera kod kojeg bi se alokacija tenzora izvršila samo jednom kod inicijalizacije sustava, a tijekom rada bi se pozivala isključivo funkcija Invoke(). Time bi se smanjilo ukupno vrijeme izvođenja i potencijalno smanjila potrošnja energije tijekom faze zaključivanja modela umjetne inteligencije.

5. Zaključak

Ovaj rad se fokusira na eksperimentalnu analizu energetskeg profila rada tipičnog radnog ciklusa uređaja ograničenih resursa. Najveći dio ciklusa sustav provodi u stanju mirovanja koje je postavljeno na jednu sekundu u ovom sustavu i troši prosječno oko 50mA. Usporedbom modela umjetne inteligencije ispostavlja se da CNN ima neznatno veću prosječnu struju otprilike 72mA u odnosu na 64mA kod RNN. Ipak, CNN faza traje znatno kraće u odnosu na RNN stoga je i potrošnja energije manja. Iznos potrošnje energije za CNN je oko 0.95mJ u odnosu na 1.9mJ kod RNN. Veće trajanje RNN modela posljedica je složenije implementacije modela koja produljuje vrijeme izvođenja. Dakle, lokalno zaključivanje ne čini sustav znatno složenijim u vidu ukupne potrošnje energije pogotovo CNN model koji se pokazao kao mali potrošač energije. To ukazuje na činjenicu da uvođenje lokalnog zaključivanja u odnosu na sustav bez lokalnog zaključivanja zaista donosi benefite. Osim što se povećava sigurnost jer se ne moraju svi podatci slati prema nadređenom sustavu koji bi onda sam obrađivao mjerene podatke vlastitim modelom umjetne inteligencije, također se može postići i ušteda energije, ako bi se nadređenom sustavu slale isključivo informacije u kojima je potrebno poduzeti neku akciju u sustavu. Na primjer, mikrokontroler može slati podatke nadređenom sustavu samo u slučaju da lokalnim zaključivanjem predvidi anomaliju u temperaturi. Tada bi bilo korisno implementirati i ekstremniju uštedu energije u fazi mirovanja u odnosu na sustav implementiran u ovom radu. Na taj način se u realnim sustavima može iskoristiti puni potencijal uvođenja lokalnog zaključivanja modela umjetne inteligencije.

Odabrani komunikacijski protokol osim na potrošnju vlastite faze utječe i na potrošnju ostalih faza u ciklusu. MQTT i UDP omogućuju Wi-Fi modemu brži ulazak u fazu štednje energije pa je prosječna potrošnja struje oko 50 mA dok TCP ostvaruje veću i promjenjiviju potrošnju jer se u svakom ciklusu ponovno stvara socket, uspostavlja TCP veza, šalje poruka i zatvara veza pa ima prosječnu potrošnju od oko 70 mA. Dakle, zaključak je da protokoli koji ne otvaraju i zatvaraju vezu kod svakog slanja i protokoli poput UDP troše znatno manje energije te da komunikacija prema

nadređenom sustavu utječe i na potrošnju ostalih faza zbog svoje asinkrone aktivnosti.

Rezultati rada pokazuju da ESP32-S3, zahvaljujući niskoj potrošnji, snažnom procesoru i integraciji s TensorFlow Lite Micro okruženjem može biti dobar izbor za implementaciju sustava koji uvodi lokalno zaključivanje modela umjetne inteligencije.

LITERATURA

Aosong (Guangzhou) Electronics Co., Ltd. Dht11 temperature humidity sensor technical data sheet. <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>.

Richard Barry. Rtos fundamentals. <https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals>, 11 2023. zadnji pristup: 9.5.2026.

Brandy. Grove - temperaturehumidity sensor (dht11). https://wiki.seeedstudio.com/Grove-TemperatureAndHumidity_Sensor/, 12 2025. zadnji pristup: 14.4.2026.

Fabio Durate. Iot statistics. <https://explodingtopics.com/blog/iot-stats>, 9 2024. zadnji pristup: 7.4.2026.

Espressif. Esp32-s3-devkitc-1. https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html, 7 2023. zadnji pristup: 3.5.2026.

Espressif. Esp32-s3 wi-fi ble 5 soc | espressif systems. <https://www.espressif.com/en/products/socs/esp32-s3>, 10 2024. zadnji pristup: 4.5.2026.

Praveen Periyasamy. Iot vs. edge vs. edge ai: Evolution and use cases. <https://www.embedur.ai/iot-vs-edge-vs-edge-ai-a-quick-compare-and-contrast/>, 9 2024. zadnji pristup: 14.4.2026.

Eksperimentalna analiza energetskeg profila rada uređaja ograničenih resursa u sustavu Interneta stvari

Sažetak

U ovom radu se provodi eksperimentalna analiza energetskeg profila rada tipičnog ciklusa uređaja ograničenih resursa temeljenog na mikrokontroleru ESP32-S3. Sustav uključuje senzor temperature DHT11 koji periodično šalje mjerenja, lokalno zaključivanje CNN i RNN modelima te komunikaciju putem UDP, TCP ili MQTT protokola. Uređaj Power Profiler Kit 2 korišten je za izračun potrošnje energije tijekom analize. CNN se pokazao energetske učinkovitijim od RNN modela unatoč nešto višoj prosječnoj struji. Ipak, kraće trajanje zaključivanja rezultiralo je manjom potrošnjom energije 0.95 mJ u odnosu na 1.90 mJ. Zaključak je da uključivanje lokalnog zaključivanja dodaje mali trošak energije, dok dominantan utjecaj na ukupnu energetske potrošnju u ciklusu imaju način upravljanja Wi-Fi štednjom i izbor komunikacijskog protokola.

Ključne riječi: Internet stvari, TinyML, ESP32-S3, računarstvo na rubu mreže, energetske profil, umjetna inteligencija, DHT11, PPK2

Experimental Analysis of the Energy Profile of Resource-Limited Devices in the Internet of Things System

Abstract

This paper presents an experimental analysis of the energy profile of a typical operating cycle of a resource-constrained device based on the ESP32-S3 microcontroller. The system includes a DHT11 temperature sensor that periodically sends measurements, on-device inference using CNN and RNN models, and communication via UDP, TCP, or MQTT protocols. The Power Profiler Kit 2 device was used to calculate energy consumption during the analysis. The CNN proved more energy-efficient than the RNN model despite a slightly higher average current. However, the shorter inference time resulted in lower energy consumption around 0.95 mJ compared to 1.90 mJ. The conclusion is that incorporating on-device inference adds a small energy cost, while the Wi-Fi power-saving management and the choice of communication protocol have a dominant influence on the total energy consumption of the cycle.

Keywords: Internet of Things, TinyML, ESP32-S3, edge computing, energy profile, artificial intelligence, DHT11, PPK2

IZJAVA O KORIŠTENJU UMJETNE INTELIGENCIJE

Tijekom izrade ovog rada korištena je umjetna inteligencija odnosno alati poput Chat-GPT (GPT-5.5) i Claude (Sonnet 4.6). Alati su korišteni kao pomoć pri učenju, traženju izvora i gramatičkih pogrešaka te programiranju.

Osvrt na korištenje umjetne inteligencije: Za poznatije i istraživanije pojmove dobivao sam uglavnom točne, precizne i korisne podatke. Ipak, za manje poznate pojmove sam morao samostalno tražiti izvore i čitati dokumentaciju jer su alati znali halucinirati i davati nepostojeće poveznice. Kod programiranja su bili od velike pomoći za traženje i ispravljanje grešaka.

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog završnog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.